Generative AI Academy

Function calling and Agents

Query BFSI customer information

This web API presents an interface to ask questions on customer information stored in a database.

user_input

Do customers who default maintain a low balance?

Clear

Submit

output

The average balance for customers who do not default is approximately 1552.84, while the average balance for customers who default is approximately -61.80.

Customers who default have a significantly lower average balance compared to customers who do not default.

What is an Agent?

Definition

An **AI agent** is an **autonomous system** that

- uses a LLM as a reasoning and decision-making core
- combined with memory, tools, and an execution loop

to perceive inputs, plan actions, and produce outputs aligned to a goal.

Perceive => parse input + context with the LLM.

Act => call tools/functions or generate text.

Achieve goals => follow user intent or multi-step objectives.

It is a **conceptual and architectural abstraction** to describe a certain *pattern of behavior* in applications built on top of LLMs.

How it maps to real world?

LLM → the reasoning core (text-in, text-out).

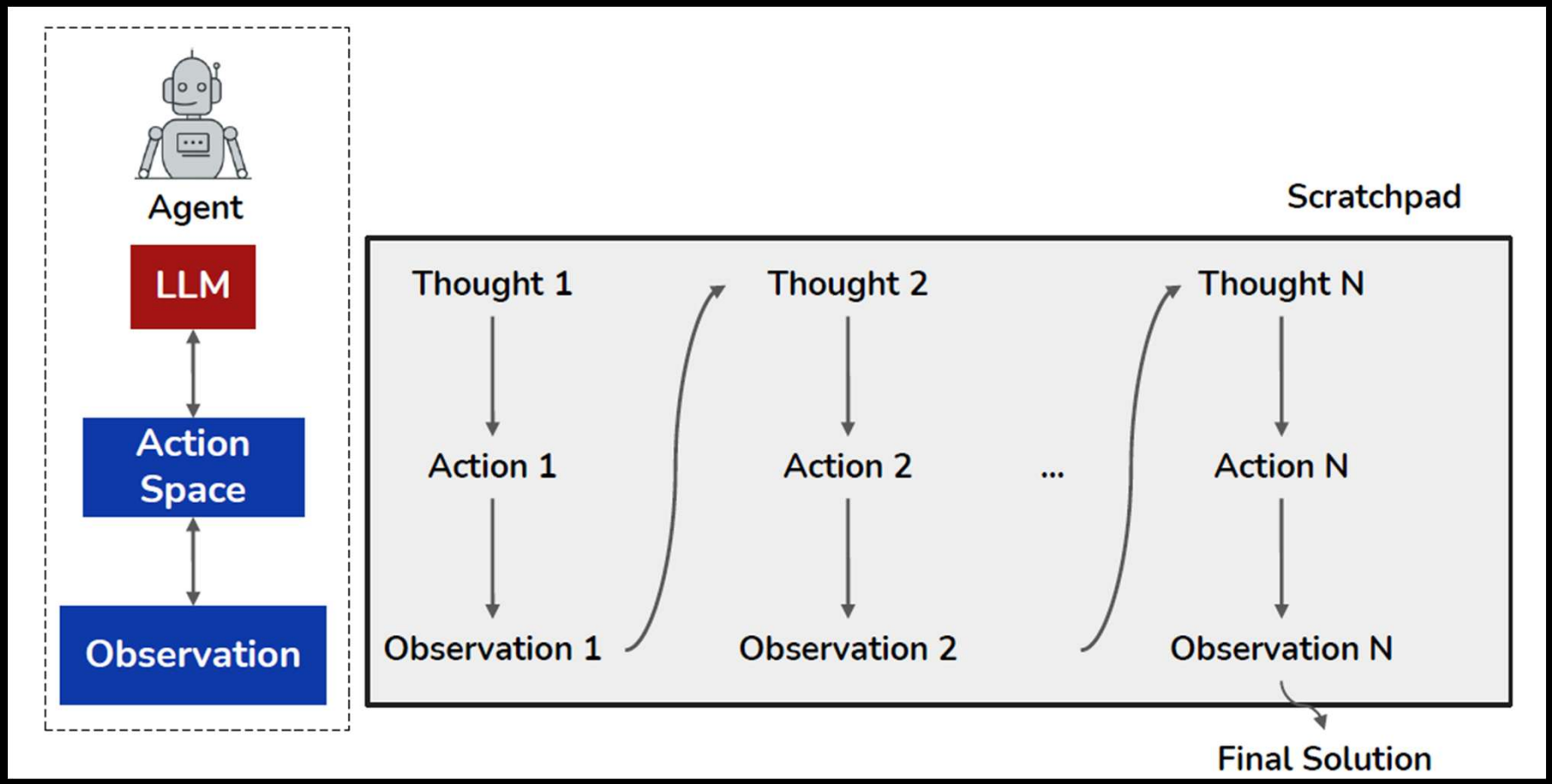
Tools/Functions → the external capabilities (APIs, databases, calculators, RAG pipelines).

Memory → persistence mechanism (conversation state, vector DB, key–value store).

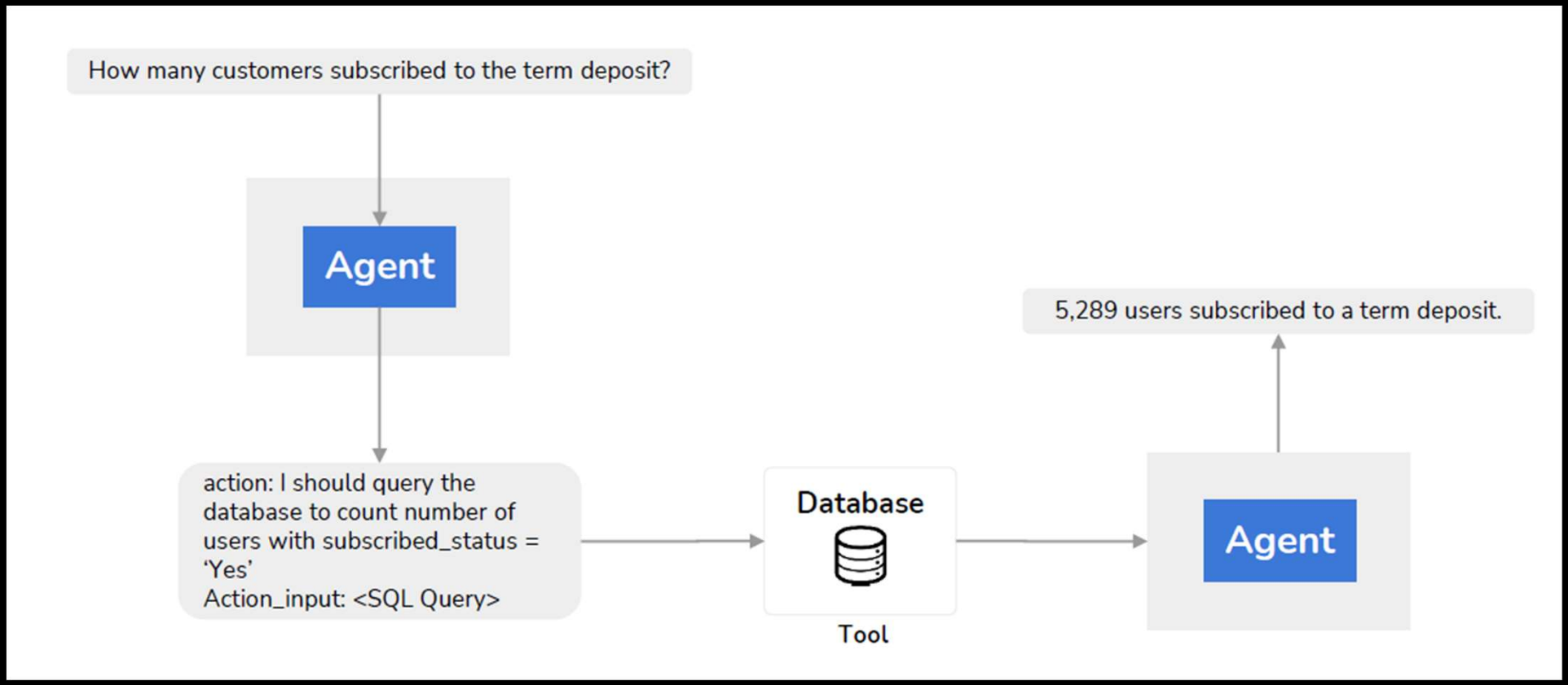
Control Loop → orchestration logic (can be as simple as ReAct prompting, or as complex as CrewAI, LangGraph, AutoGen, etc.).

Put together, **the whole application that stitches these pieces together is what we call an "agent."**

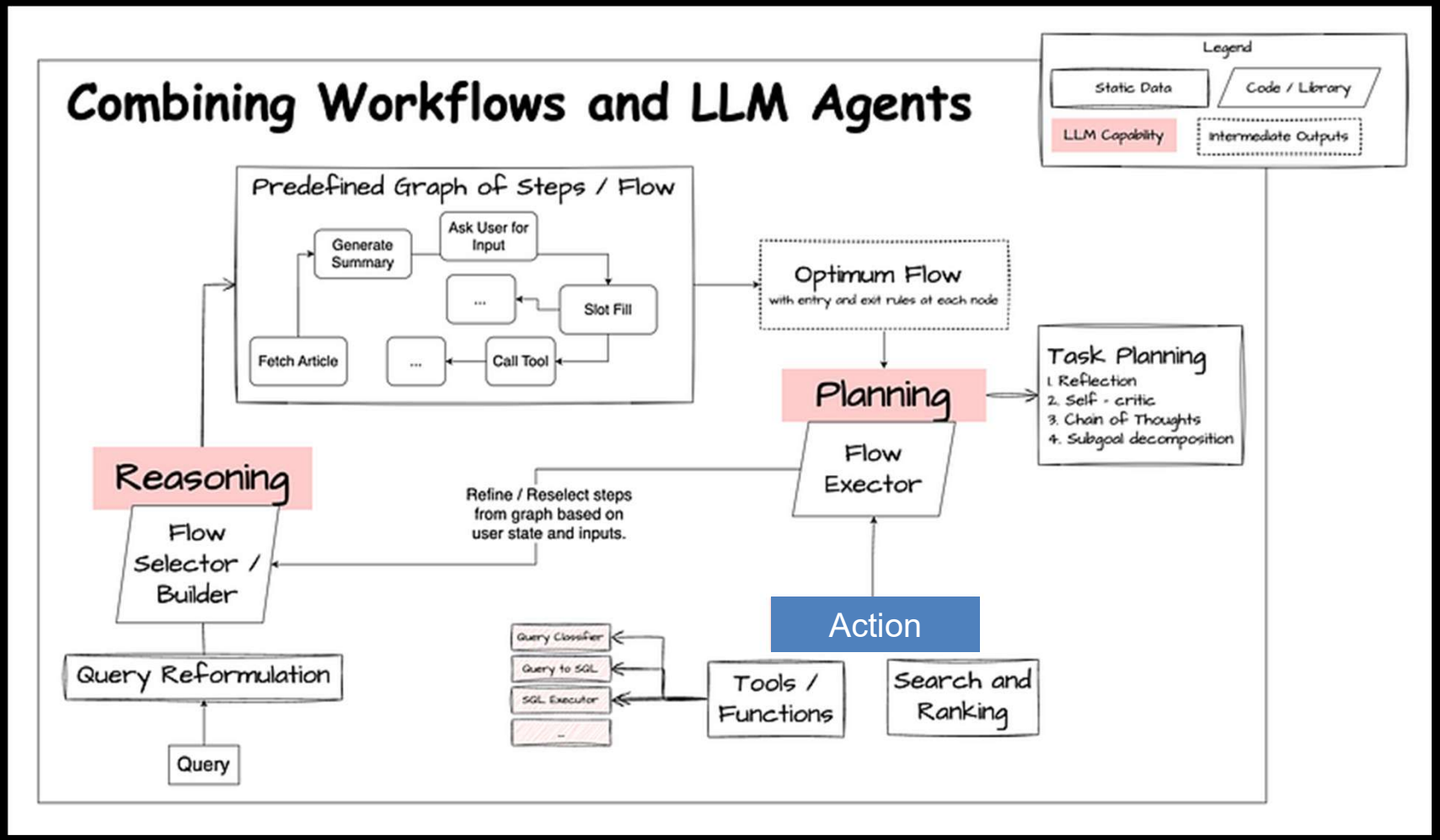
ReAct paradigm (Reasoning and Action)



Agent workflow

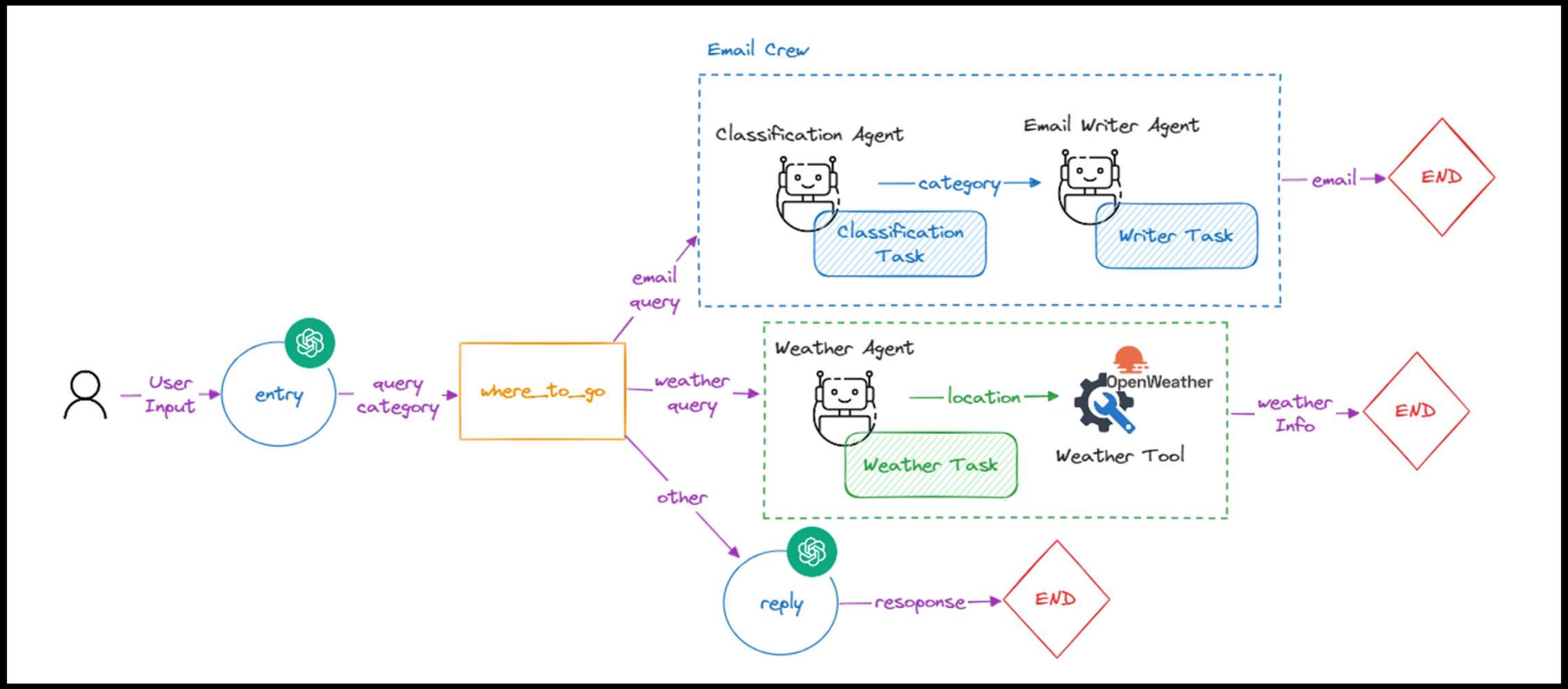


Workflow with Agents – Agentic AI



https://medium.com/@raunak-jain/path-to-production-for-llm-agents-f0a9cafd2398

Workflow example



Function calling and Agents
Demos and hands-on

Frameworks to build Agentic Apps

Why are frameworks needed?

- Simplify workflow orchestration of multi agent flows
- Inter agent communication
- Ability for Agents to have a **state**
- Memory
  - Short term
  - Long term
  - Episodic (conversation boundaries / topic boundaries / task boundaries)

Popular Agentic Frameworks

Code Based

- Langchain Agents and Langgraph
- LLamaIndex Tools
- Open AI Swarm
- Crew AI
- And many more (RASA, AutoGen, SmolAgents etc.)

Low code/No code:

- n8n
- Flowise
- Amazon Bedrock Flows (https://aws.amazon.com/bedrock/flows/)
- Azure Agent Builder (https://learn.microsoft.com/en-us/power-apps/maker/canvas-apps/agent-builder)

Considerations

- Langgraph, Crew, Swarm offer powerful orchestration and workflow components
- Application design considerations will involve properly marrying these capabilities with LLM capabilities around summarization, generation, reasoning and planning

- Rule based workflows are perhaps better captured by hard wiring the flow using these tools for predictability and reliability

Crew AI

Primary Functionality

CrewAI is built around the concept of a "crew" of agents working together. It uses a **structured process flow** to execute tasks sequentially or in parallel

Customization

Offers role-based specialization, where agents are defined by roles and goals. This framework abstracts complexity by providing high-level constructs for task management.

Workflow Management

Uses a **sequential execution model** by default, but can be scaled by running multiple crews in parallel.

Ease of Use

Known for excellent documentation and a robust community, making it easier for users to get started

LangGraph

LangGraph models agentic workflows as a state machine (a graph), allowing for explicit control over loops, edges, and state transitions.

LangGraph's fundamental architecture is a **state graph**, making it ideal for processes that are not strictly sequential and require loops (cycles).
While other frameworks can simulate this, LangGraph provides a more natural and robust way to define and manage such complex, state-dependent flows.

LangGraph uses typed state objects that are passed between nodes and updated immutably. This provides structured, type-safe state management that persists across the entire graph execution, making it ideal for complex multi-step workflows.

It also provides a visual tool, LangGraph Studio to build such graphs visually

Microsoft AutoGen

- Opensource library for building Agentic Applications - Microsoft is primary contributor
- Multi-agent workflows with asynchronous, event-driven architecture
- Support for both Python and .NET programming languages
- Customizable and conversable agents which integrate LLMs, tools, and humans
- AutoGen Studio - a low-code interface for building multi-agent workflows

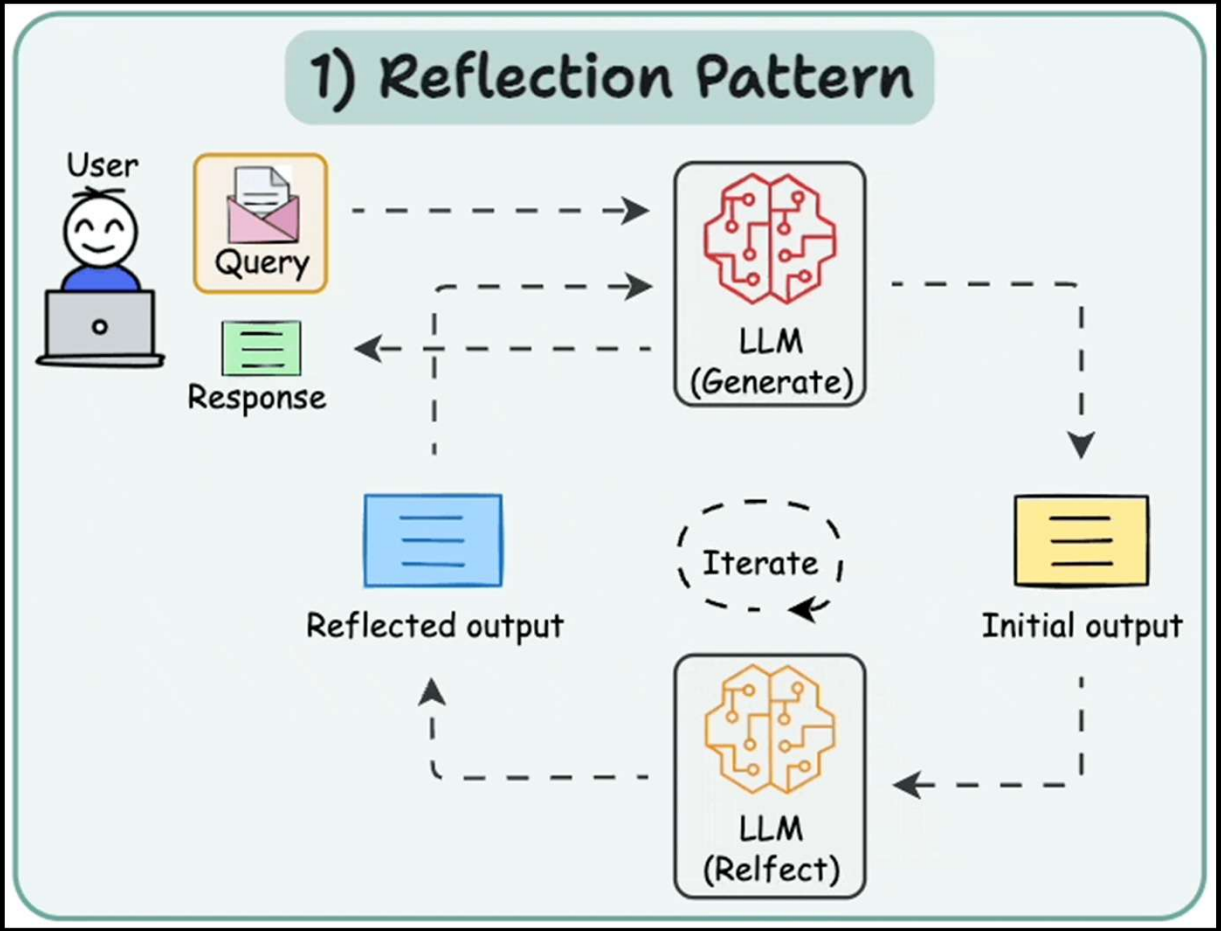
A2A (Agent to Agent)

- A2A is a communication protocol, not a framework or library.
- Released by Google in Apr 2025, open-sourced and released to Linux Foundation in Jun 2025
- Designed so agents (regardless of their internal codebase or host framework) can send secure, interpretable requests and responses to one another—for example, over HTTP using JSON-RPC 2.0.
- Enables agents built in different systems (AutoGen, LangGraph, proprietary) to collaborate without manual integration.
- Handles authentication, authorization, and streaming updates; acts as a “universal translator” for agents.

Demo of sample applications with LangGraph, Crew AI

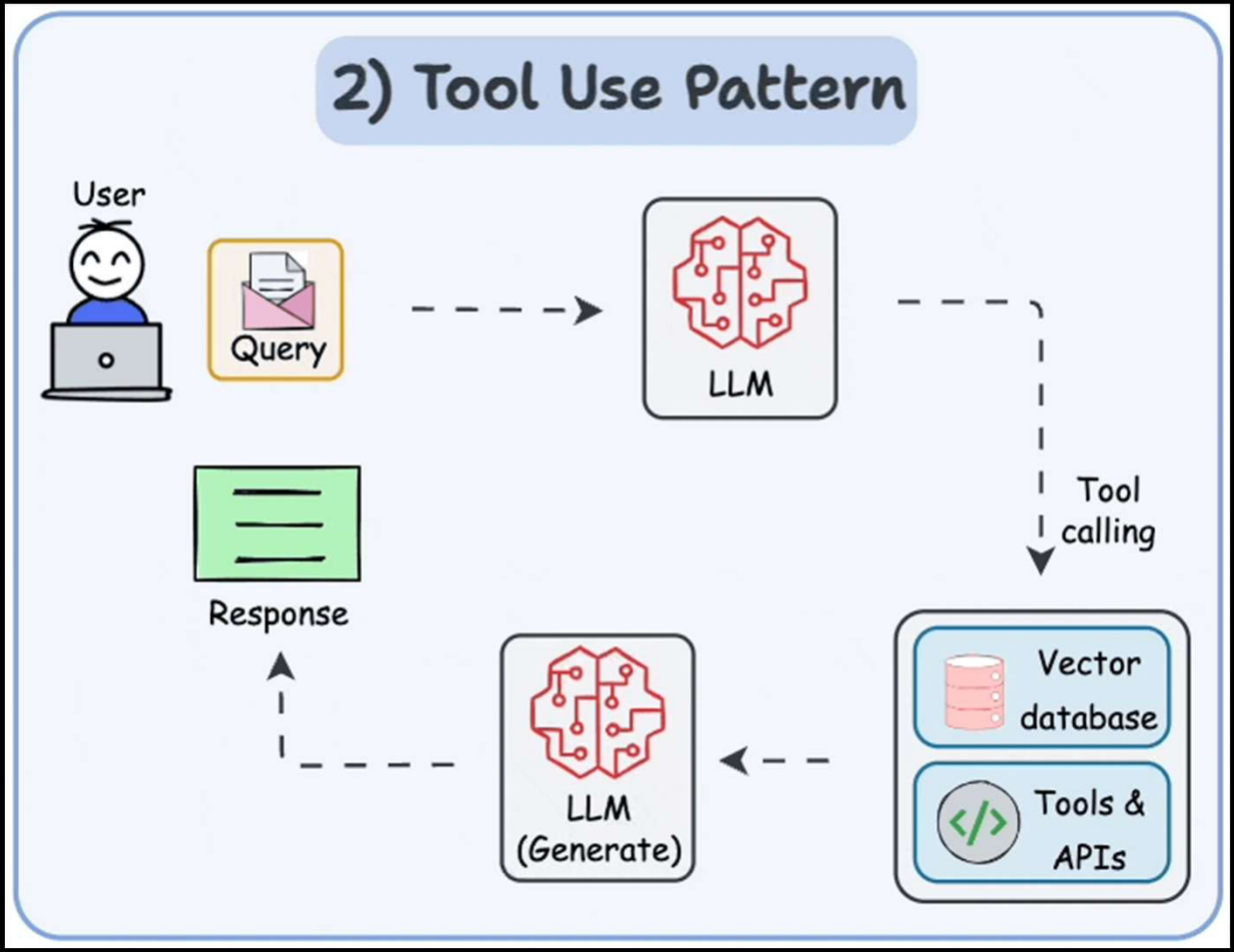
Agentic AI Design Patterns

Reflection

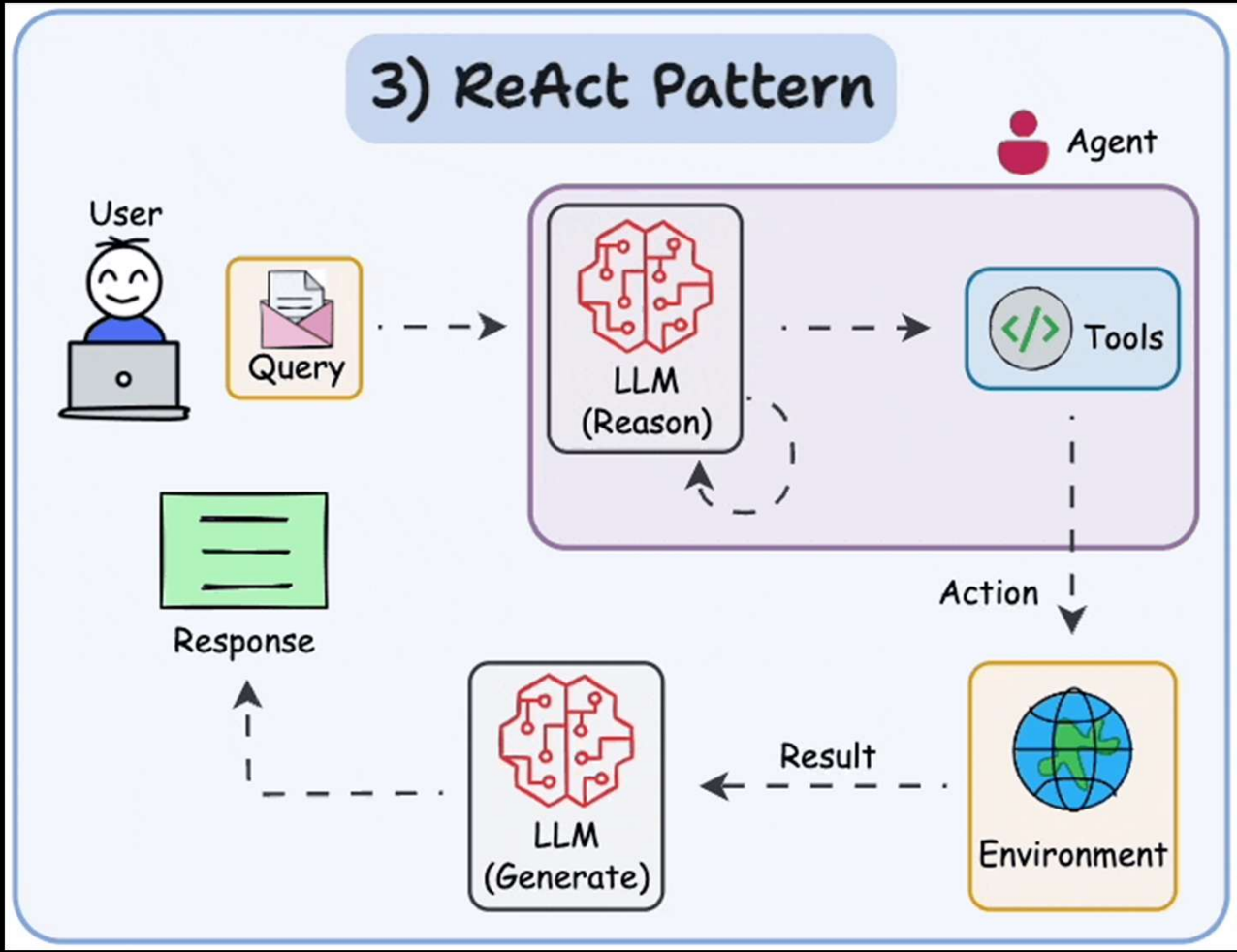


https://medium.com/data-science-collective/stop-prompting-start-designing-5-agentic-ai-patterns-that-actually-work-a59c4a409ebb

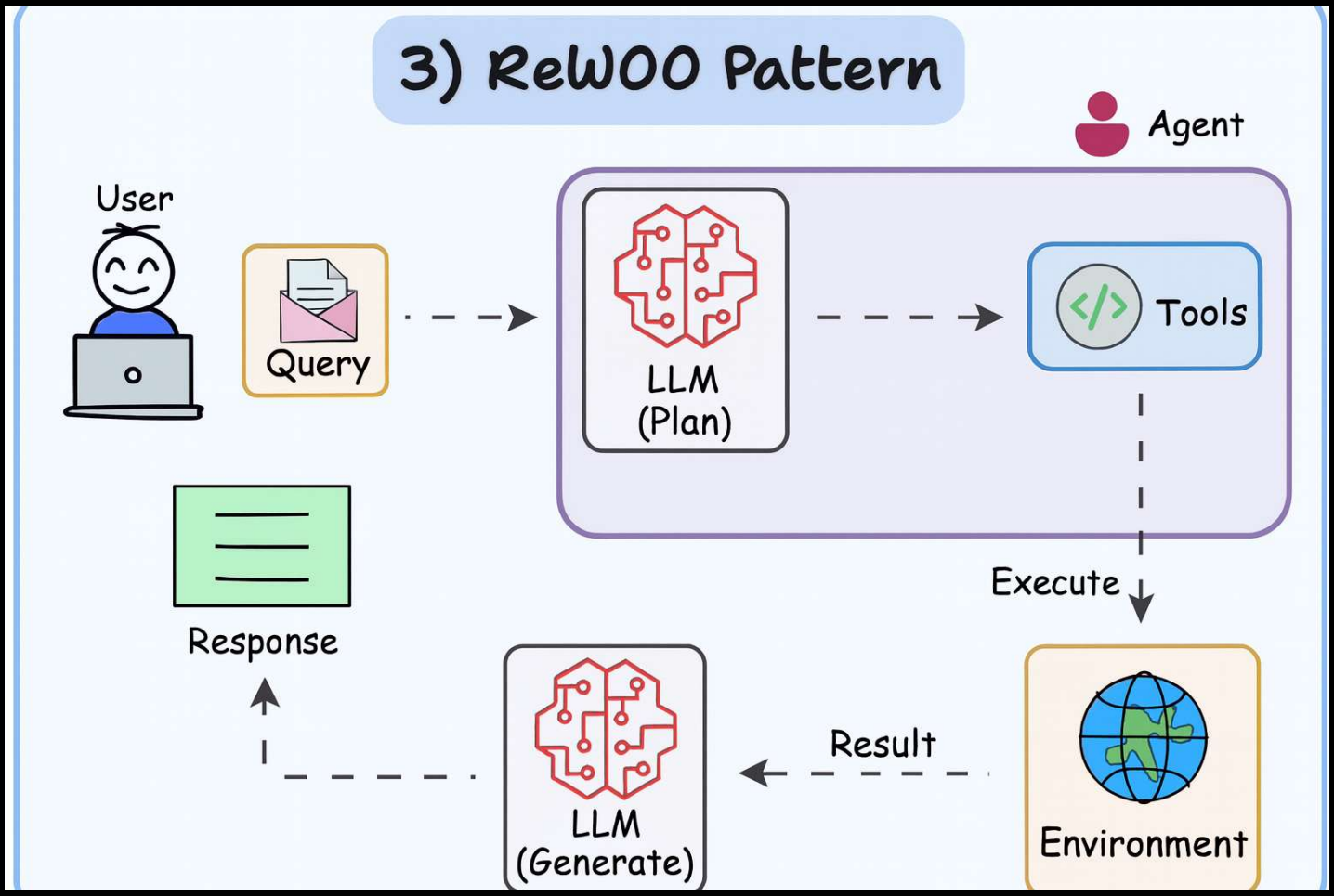
Tool Use


2) Tool Use Pattern
User
Query
LLM
Tool calling
Vector database
Tools & APIs
LLM (Generate)
Response

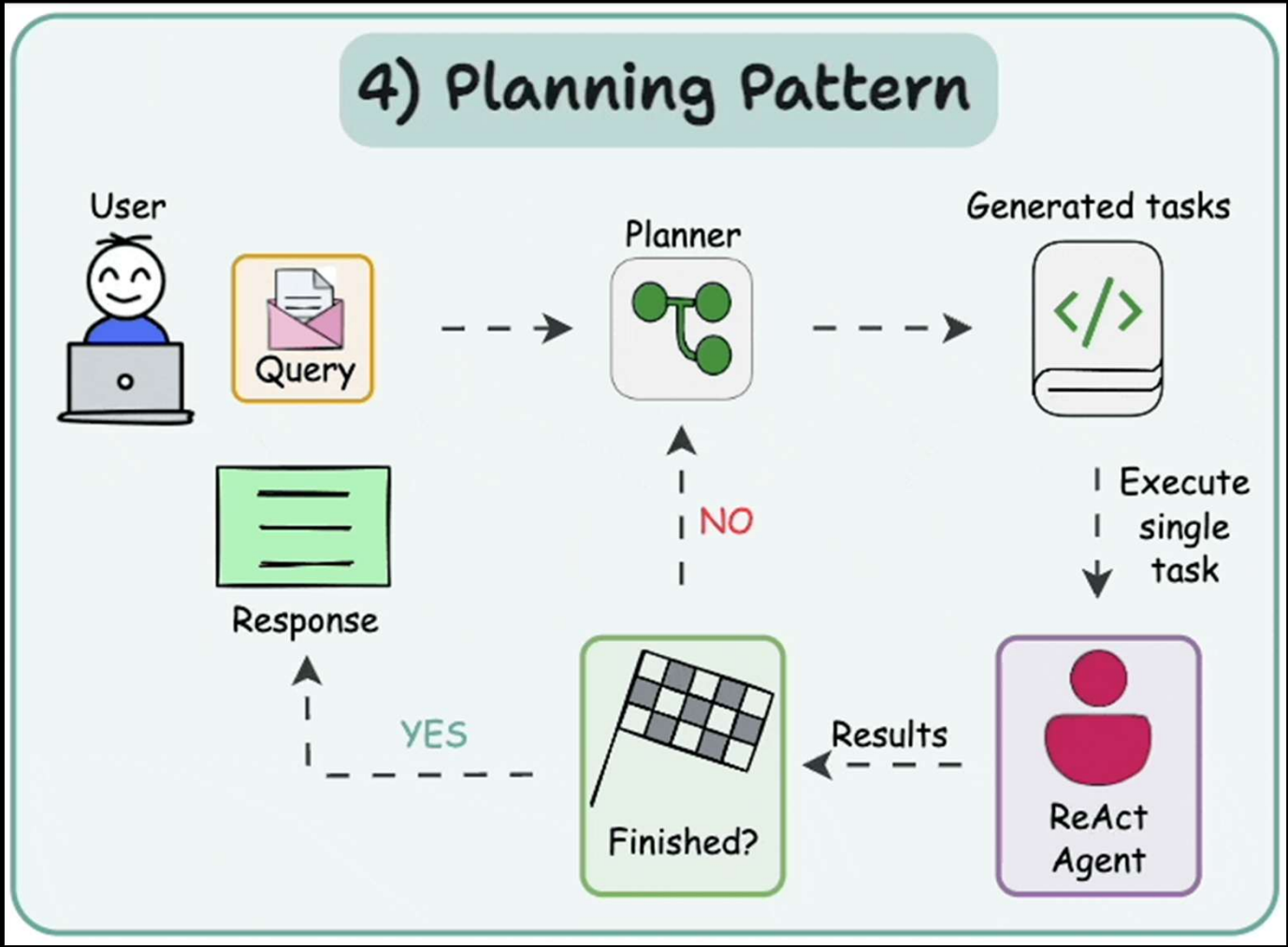
ReAct


3) ReAct Pattern
Agent
User
Query
LLM
(Reason)
Tools
Action
Response
LLM
(Generate)
Result
Environment

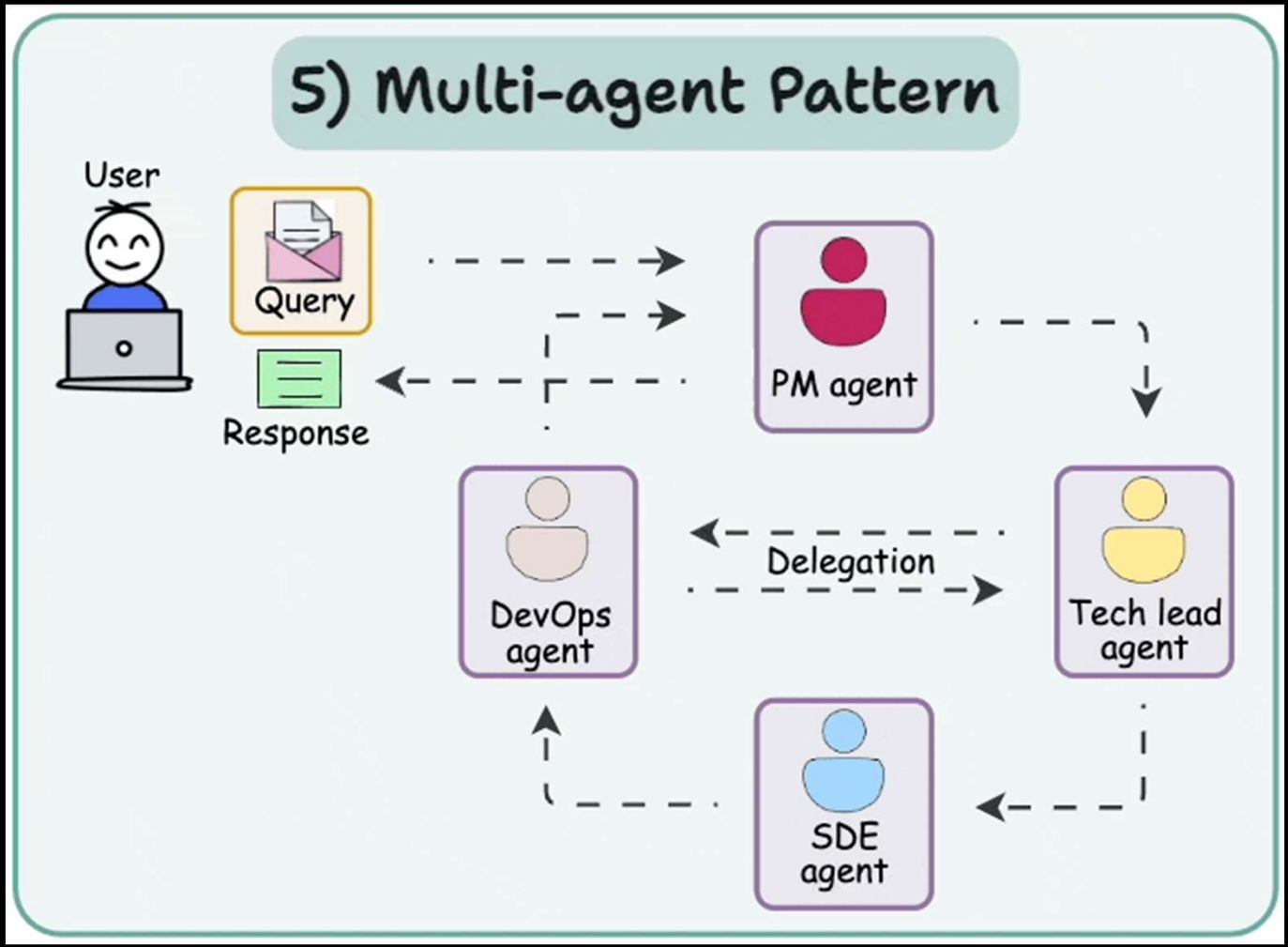
ReWoo (Reasoning Without Observation)



Planning Pattern



Multi agent – collaborative with memory and state



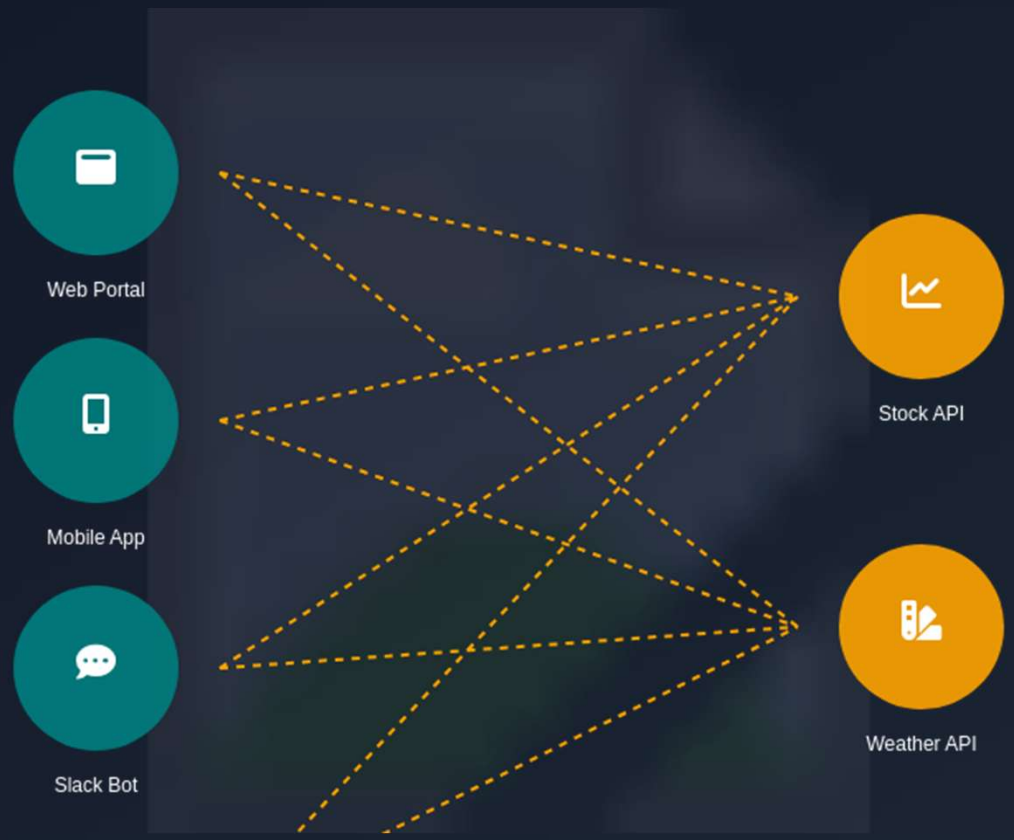
Demos of Agentic AI Design Patterns

MCP: Model Context Protocol

Models are only as good
as the **context** provided to them.
— Anthropic

M x N Problem

The complexity of point-to-point connections



8 connections

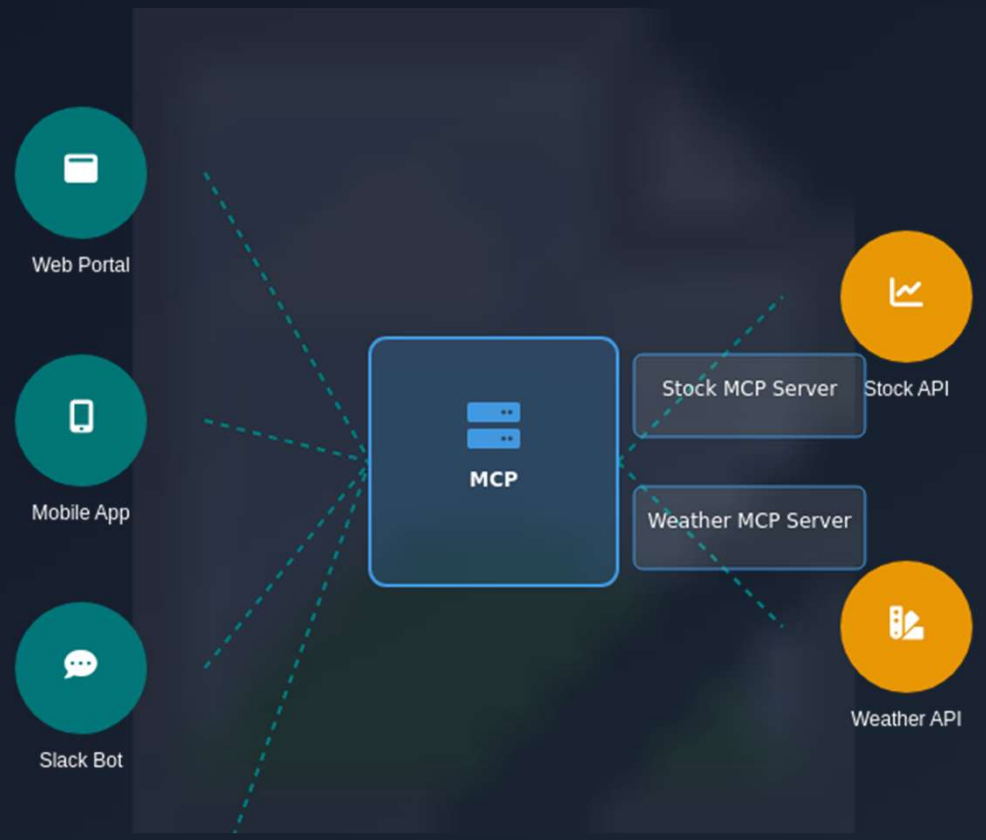
The M×N Problem

- 4 applications × 2 services = **8 connections**
- Each connection requires separate SDKs, authentication, error handling
- Different authentication flows: API keys, OAuth, etc.
- Custom response mapping for each service

*Each new application or service **exponentially increases** integration*

The MCP Solution: Standardized Integration

Transforming M×N to M+N with standardized clients and servers



6 connections

The M+N Solution

- 4 applications + 2 services = **6 connections**
- Each app uses a standard MCP client interface
- Each service is wrapped in a standardized MCP server
- Uniform protocol for all connections

*New applications or services only require **one new connection** to the MCP ecosystem*

What is MCP?

The **Model Context Protocol** standardizes how **AI applications** and **agents** interact with external **tools** and **data sources.**

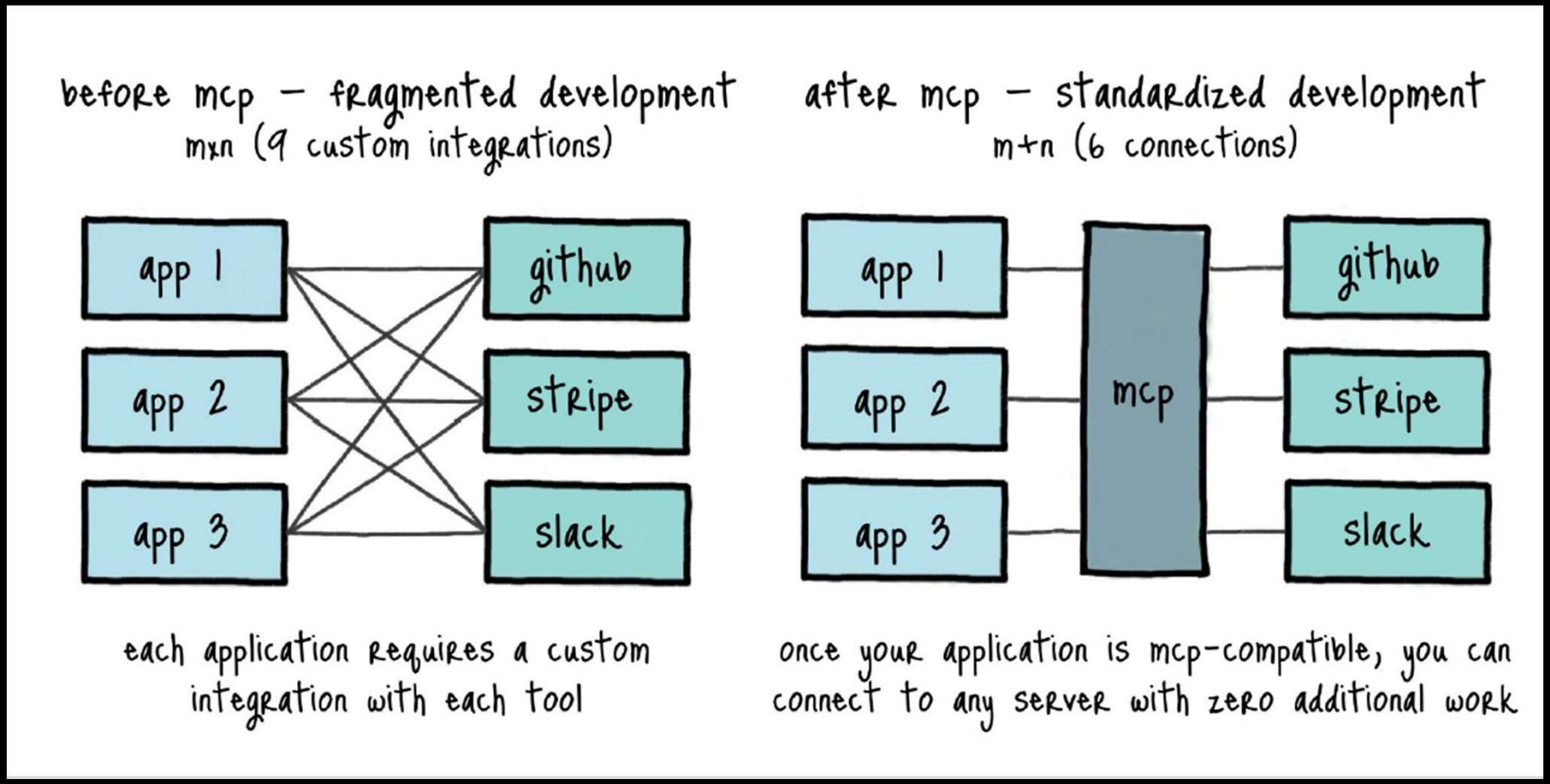
MCP is a **low-level application protocol** (the “USB-C of AI apps”) that standardizes connectivity between any App/Agent/Tool and any external tool or data source.

It’s a **specification** you build against, with SDKs and server/client reference implementations

MCP – Standardized the interface

MCP acts like a “USB port”, providing AI applications with a **universal interface** to access **relevant context** and **perform actions** on other systems.

From fragmented to standardized



MCP – server side simplicity

Any team or vendor can spin up an MCP server that exposes one or more capabilities

- GitHub-connector
- Postgres-connector
- shell-executor, etc.

The servers speak the same JSON-RPC wire protocol, so any MCP-aware client can plug in.

MCP – client side simplicity

From the client's perspective

- Just point your MCP SDK at the server's URL (and credentials) and immediately gain all its functions.
- No per-tool SDKs
- No custom wiring
- Enables discovery

Real World scenario – Finance portfolio Q&A App

Scenario:

Your portfolio-management team in a private-bank wants an on-demand assistant that can answer natural-language questions about client portfolios, flag any trades that may breach the bank's risk limits, and generate quick compliance memos.

How MCP helps?

- Data Connectors
  - Postgres/snowflake/sharepoint/S3

- Uniform client SDK
    - Agents use the MCP client APIs to discover and access the services in a uniform manner

Is MCP Server offering same functionality as API Gateways?

Traditional API Gateway / Backend	MCP Server Layer
✅ Full control over routing, auth, rate-limits, SLAs	✅ Leverages that control: proxies through your gateway, reusing the same OAuth/JWT flows
✅ Battle-tested middleware (logging, metrics, retries)	✅ Inherits all of the above—MCP sits on top of your existing services
❌ Requires custom client libraries or code-gen per service	✅ Provides a one-size-fits-all JSON-RPC client schema consumable by any MCP-aware agent
❌ No native "tool discovery" for LLMs	✅ Agents can introspect available methods, parameters and docs at runtime

MCP specification: communication protocol

It uses **JSON-RPC 2.0** as the transport and message format.

All requests and responses (e.g. tools/list, tools/call, resources/list) follow JSON-RPC semantics:

Request from MCP Client

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/list",
  "params": {}
}
```

Response from MCP Server

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "tools": [
      {
        "name": "get_weather",
        "description": "Get current weather for a city",
        "inputSchema": { "type": "object", "properties": { "city": { "type": "string" } } }
      }
    ]
  }
}
```

Creating an MCP Server

Python and Javascript/Typescript are popular languages supported

Node.js and Python packages are available that enable creating our own MCP Servers.

Key parts:

- Metadata of the tools: name, description, parameters, parameter descriptions
- Implementation of the tools
- Implementation of standard interfaces such as tools/list, tools/call
- Authentication and Authorization mechanisms

Exposing MCP tools via web service/http

Streamable HTTP transport enables exposing these tools as an HTTP Endpoint

FastMCP is an example of such a server implementation

Web Services such as Zapier MCP provide an easy mechanism to expose various tools as an MCP Endpoint

https://mcp.zapier.com

Using the tools in a client application

No-code integration with any client application that supports MCP

- Any Client application that supports MCP can use these tools without any coding
- Popular AI chatbots such as Claude, ChatGPT and IDEs such as Cursor, VS Code support MCP
- When given an MCP Endpoint, they can discover the tools and add them to your chatbot

Programmatic consumption in applications

- Python and Javascript libraries available to simplify programmatic discovery and integration of tools into any application

MCP and LLM

- MCP is a tools standardization specification, and hence does not directly talk to the LLM
- Similarly, LLM is MCP Agnostic

Then what does it have to do with Agents/Agentic AI?

- MCP helps as the tools can now be discovered by the application and passed onto the LLM
- Similarly, the function calls requested by the LLM can directly be sent to MCP Server for execution
- Hence the Application code becomes minimalistic, clean and there is good separation of concerns

So in our get_weather example, the Application will not implement any get_weather, It will:

a) Discover get_weather via MCP Server(s) (via tools/list)
b) Pass the metadata to the LLM as tools/functions
c) When requested by the LLM, invoke the tool in the MCP Server (via tools/call)

MCP and Agentic frameworks such as crew.ai

Agentic AI frameworks such as crew.ai provide good integration and capabilities to use MCP

```
# MCP tools become regular CrewAI tools
with MCPServerAdapter(server_params=...) as mcp_adapter:
    weather_tools = mcp_adapter.get_tools()  # MCP → CrewAI tools

    agent = Agent(
        role='Weather Expert',
        tools=weather_tools,  # Agents see these as normal tools
        # Agent has no idea these come from MCP!
    )
```

Code Demos of MCP capabilities

Agentic AI low code / no code visual builders

n8n

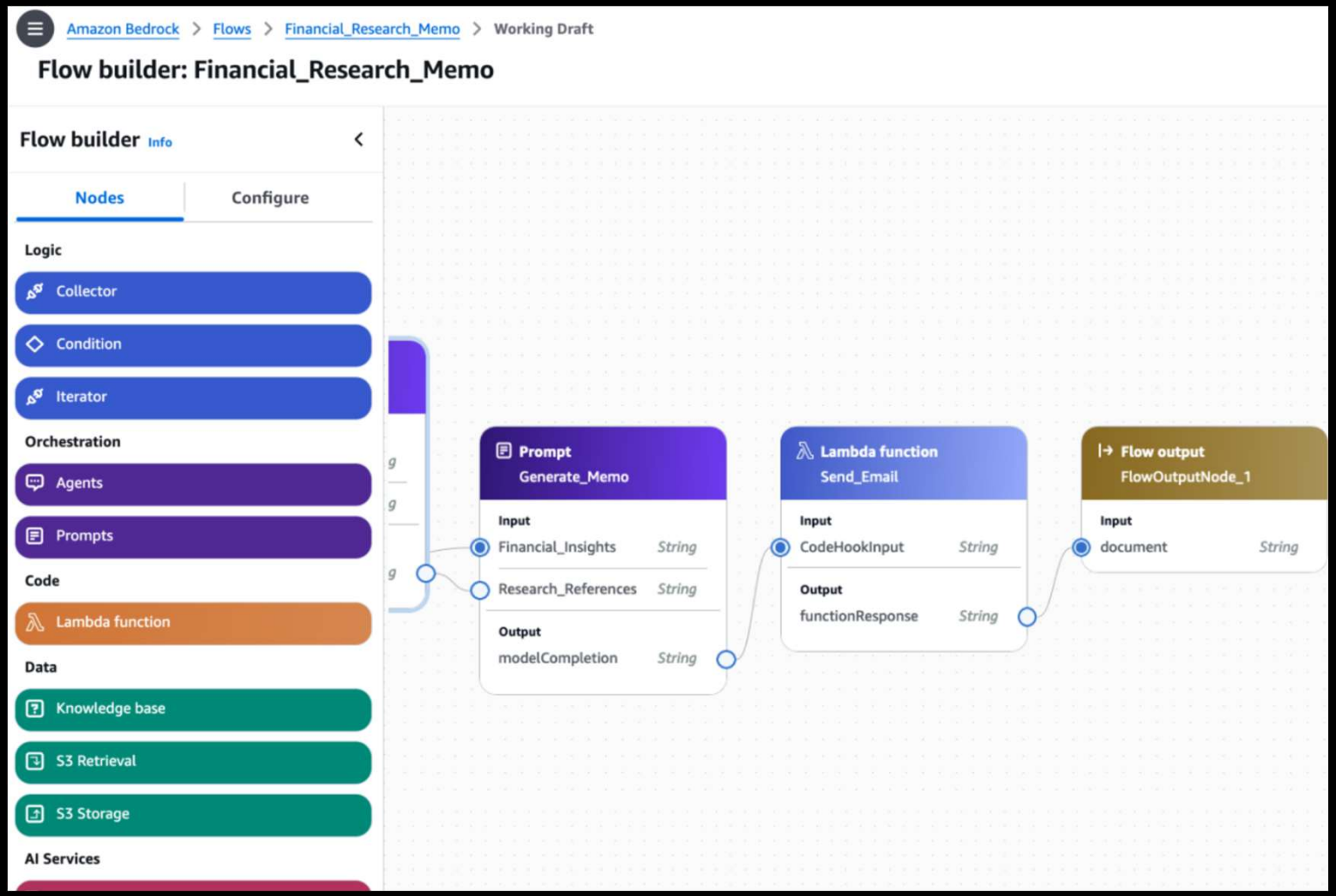
- n8n is an open-source, "fair-code" workflow automation platform that combines traditional workflow automation with advanced AI capabilities, specifically focusing on agentic AI.
- It allows users to visually design complex AI-powered workflows using a node-based interface, blending AI models, pre-defined logic, custom code, and human-in-the-loop interventions.

N8n – the company

- n8n is a company founded in 2019 by **Jan Oberhauser**.
- It operates with a "fair-code" distribution model, meaning its source code is visible and can be self-hosted, allowing for custom functions, logic, and apps.
- The company has secured significant funding rounds from investors including Sequoia, Felicis Ventures, and Highland Europe.

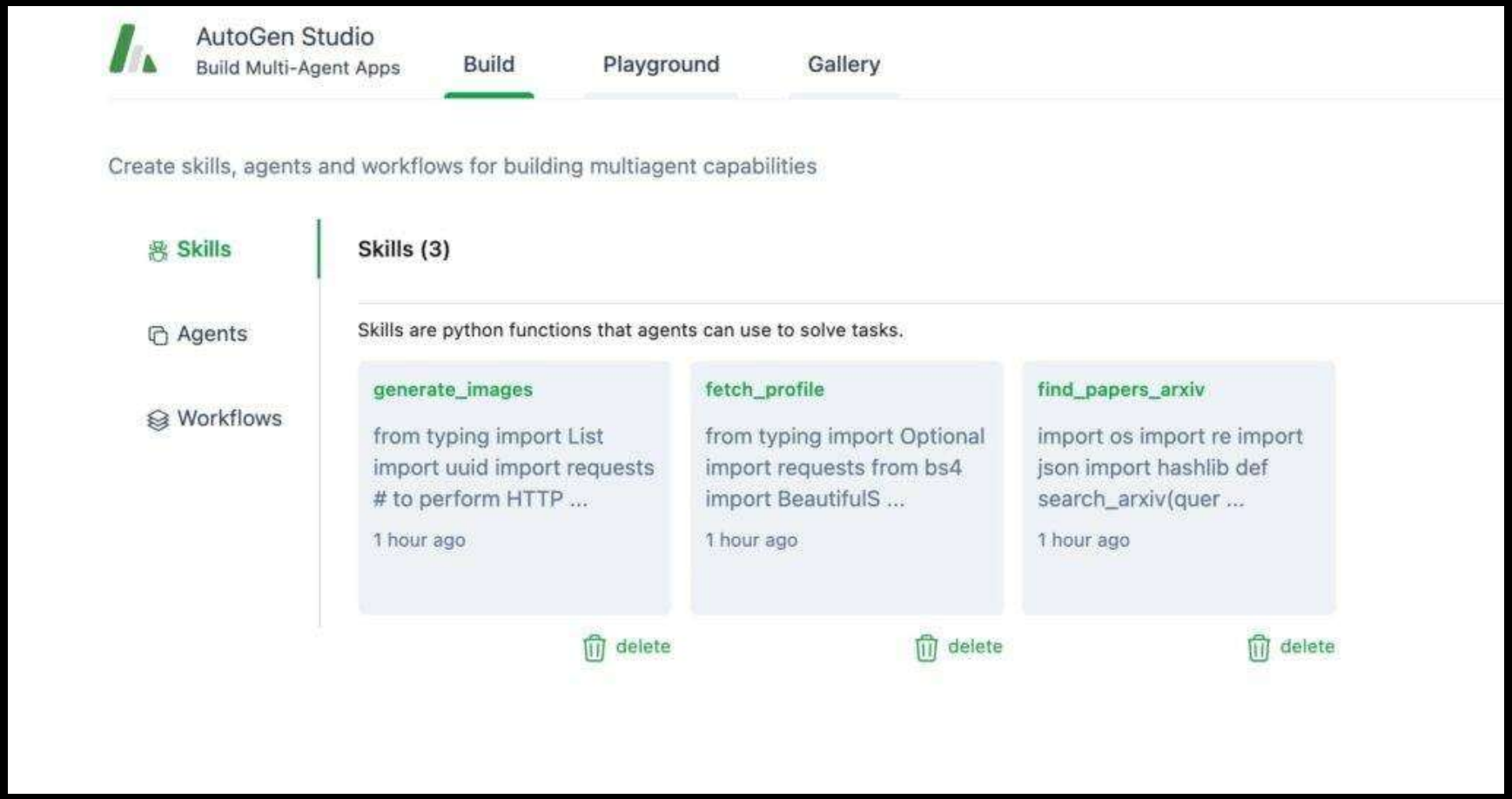
https://blog.n8n.io/ai-agentic-workflows/

Amazon Bedrock Flows



https://aws.amazon.com/bedrock/flows/

Microsoft Azure AutoGen



https://www.microsoft.com/en-us/research/blog/introducing-autogen-studio-a-low-code-interface-for-building-multi-agent-workflows/
https://ai.gopubby.com/autogen-studio-ui-2-0-a-step-by-step-guide-for-beginners-33c21e801c45

Demo of Visual Agentic Workflow builder with n8n